

ChronoPath

Structure du projet / Guide des dossiers et fichiers

HEIG-VD · Projet GIN 2026

1. Arborescence complète

Voici la structure complète du projet. Chaque dossier et fichier est détaillé dans les sections suivantes.

```
GIN_projet/
├── backend/
│   ├── main.go
│   ├── server.exe           (généré par go build)
│   ├── go.mod
│   ├── go.sum
│   ├── .env
│   └── handlers/
│       ├── db.go
│       ├── routing.go
│       ├── geodata.go
│       ├── pois.go
│       └── proxy.go
├── frontend/
│   ├── index.html
│   ├── css/
│   │   └── style.css
│   ├── images/
│   │   └── logo2.png
│   └── js/
│       ├── map.js
│       ├── cantons.js
│       ├── trails.js
│       ├── search.js
│       ├── routing.js
│       ├── app.js
│       └── create.js
├── Donnees_finales/
│   └── CANTONS_Romands.gpkg
└── README.md
```

2. Dossier backend/

Contient tout le code serveur Go. Le serveur écoute sur le port 8080 et expose les fichiers statiques du frontend ainsi que l'API REST.

2.1 Fichiers racine

Fichier	Type	Rôle
<code>main.go</code>	<i>Go</i>	Point d'entrée du serveur. Initialise Gin, déclare toutes les routes API (<code>/api/route</code> , <code>/api/search-routes</code> , <code>/api/osm-ways...</code>) et sert les fichiers statiques du frontend (HTML, CSS, JS, images).
<code>go.mod</code>	<i>Go</i>	Déclare le module Go et ses dépendances directes (Gin, lib/pq pour PostgreSQL, godotenv pour <code>.env</code>).
<code>go.sum</code>	<i>Go</i>	Fichier de vérification des dépendances (checksums). Généré automatiquement par <code>go mod tidy</code> . Ne pas modifier manuellement.
<code>.env</code>	<i>Config</i>	Variables de connexion à la base de données : <code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASS</code> . Chargé automatiquement au démarrage via <code>godotenv</code> .
<code>server.exe</code>	<i>Binaire</i>	Binaire compilé par <code>go build -o server.exe</code> . Non versionné dans git. À régénérer après chaque modification Go.

2.2 Dossier handlers/

Chaque fichier handler correspond à un groupe de fonctionnalités de l'API. Gin appelle ces fonctions lors des requêtes HTTP.

Fichier	Type	Rôle
<code>db.go</code>	<i>Go</i>	Initialise la connexion à PostgreSQL via <code>database/sql</code> et <code>lib/pq</code> . Expose la variable globale DB utilisée par tous les autres handlers. Lit les credentials depuis les variables d'environnement (<code>.env</code>).
<code>routing.go</code>	<i>Go</i>	Cœur du système de routage. Contient <code>SearchRoutes()</code> pour la recherche d'itinéraires (boucles triangulaires, mode destination) et <code>CalculateRoute()</code> pour le calcul A→B. Implémente l'algorithme Dijkstra via <code>pgr_dijkstra</code> et le calcul de vitesse selon FTP/VAP/niveau.
<code>geodata.go</code>	<i>Go</i>	Endpoint GET <code>/api/osm-ways</code> . Retourne les routes OSM dans un bounding box donné depuis la table <code>ways</code> de <code>pgRouting</code> . Limité à 2000 éléments pour les performances.
<code>pois.go</code>	<i>Go</i>	Endpoint GET <code>/api/pois</code> . Gestion des points d'intérêt (structure prévue pour extension).
<code>proxy.go</code>	<i>Go</i>	Proxy pour requêtes externes si nécessaire (WMS, tuiles). Évite les problèmes CORS lors de l'appel à des services tiers depuis le frontend.

3. Dossier frontend/

Contient l'intégralité de l'interface utilisateur. Les fichiers sont servis directement par le backend Go via `r.Static()` et `r.StaticFile()`. Aucun build frontend n'est nécessaire.

3.1 Fichier principal

Fichier	Type	Rôle
<code>index.html</code>	<i>HTML</i>	Page unique de l'application (SPA). Contient la structure complète de la sidebar (4 onglets : Rechercher, Créer, Sauvegardés, Itinéraire), les panneaux de filtres pour chaque sport (Vélo, Rando, Course), le panneau carte et tous les éléments UI. Charge tous les scripts JS et CSS.

3.2 Dossier css/

Fichier	Type	Rôle
<code>style.css</code>	<i>CSS</i>	Feuille de styles principale. Définit les variables CSS (<code>--green</code> , <code>--bg</code> , <code>--border...</code>), les styles de la sidebar, des onglets, des sliders, des cartes de résultats, des boutons, des popups et de la carte Leaflet. Contient aussi les styles responsive.

3.3 Dossier images/

Fichier	Type	Rôle
<code>logo2.png</code>	<i>Image</i>	Logo de l'application affiché dans l'en-tête de la sidebar. Chargé via <code></code> .

3.4 Dossier js/ — ordre de chargement

Les scripts sont chargés dans un ordre précis en bas du `<body>` pour garantir que les dépendances sont disponibles. `map.js` doit être chargé en premier car il crée la variable globale `map` utilisée par tous les autres scripts.

Fichier	Ordre	Rôle
<code>map.js</code>	<i>1er</i>	Initialise la carte Leaflet et crée la variable globale <code>map</code> . Configure les deux fonds de carte Swisstopo (Topographique et Orthophoto) via WMTS. Gère le contrôle de couches. Doit être chargé en premier.
<code>cantons.js</code>	<i>2e</i>	Gère la couche des cantons romands (chargement du GeoPackage converti en GeoJSON), les routes OSM dynamiques (rechargement automatique au <code>moveend/zoomend</code>), et la légende des types de routes. Dépend de <code>map</code> .
<code>trails.js</code>	<i>3e</i>	Définit le tableau <code>TRAILS</code> avec les itinéraires prédéfinis (vélo, rando, course). Contient les fonctions <code>renderTrails()</code> , <code>filterTrails()</code> , <code>showDetail()</code> , <code>exportGPX()</code> , <code>saveTrail()</code> . Les itinéraires ne sont PAS affichés au chargement (<code>renderTrails</code> est commenté dans <code>app.js</code>).

<code>search.js</code>	4e	Gère la fonctionnalité Rechercher. Appelle l'API <code>/api/search-routes</code> avec les paramètres FTP, VAP, niveau, durée. Affiche les 3 résultats sur la carte avec popups et dans la liste. Gère l'export GPX des résultats.
<code>routing.js</code>	5e	Gère l'onglet Itinéraire A→B. Picking de points sur la carte (<code>startPicking</code>), géocodage Nominatim, calcul Dijkstra via <code>/api/route</code> , affichage du tracé, étapes intermédiaires multiples (<code>addWaypointField</code>), chargement GPX, export GPX.
<code>app.js</code>	6e	Script d'initialisation principal. Gère les onglets sport et sous-onglets, les sliders (mise à jour des valeurs affichées), les boutons filtres, le point de départ (géocodage, GPS, clic carte), et la connexion entre l'UI et les fonctions des autres scripts.
<code>create.js</code>	7e (dernier)	Gère les onglets Créer et Sauvegardés. Mode tracer (points sur carte), calcul pgRouting segment par segment, sauvegarde localStorage. Import GPX (extraction dénivelé depuis <code><ele></code>), affichage/modification/suppression des items sauvegardés, calculateur de temps (FTP/VAP/Naismith), export GPX.

4. Dossier Donnees_finales/

Fichier	Type	Rôle
<code>CANTONS_Romands.gpkg</code>	<i>GeoPackage</i>	Polygones des cantons romands au format GeoPackage (Swissstopo). Converti en GeoJSON côté backend pour être servi au frontend. Utilisé pour afficher les limites cantonales sur la carte.

5. Fichier README.md

Documentation principale du projet à la racine. Contient les instructions de démarrage rapide (docker start, go build), l'installation initiale, la description de tous les endpoints API, les fonctionnalités, les formules de calcul de vitesse et un tableau de dépannage.

6. Variables globales importantes

Ces variables JavaScript sont accessibles depuis tous les scripts via l'objet window :

Variable	Définie dans	Description
<code>map</code>	<i>map.js</i>	Instance Leaflet principale. Utilisée par tous les autres scripts pour ajouter/supprimer des couches.
<code>API_BASE</code>	<i>app.js</i>	URL de base de l'API (<code>http://localhost:8080/api</code>). Utilisée dans tous les <code>fetch()</code> .
<code>currentSport</code>	<i>app.js</i>	Sport actuellement sélectionné : 'velo', 'rando' ou 'course'.
<code>searchStartCoords</code>	<i>app.js</i>	Coordonnées {lat, lon} du point de départ sélectionné pour la recherche.
<code>routeStart / routeEnd</code>	<i>routing.js</i>	Points de départ et d'arrivée pour le calcul A→B.

<code>savedItems</code>	<code>create.js</code>	Tableau des itinéraires sauvegardés. Persisté dans localStorage sous la clé 'chronopath-saved'.
<code>layers</code>	<code>app.js</code>	Objet contenant les LayerGroups Leaflet : layers.cantons, layers.routes, layers.search, layers.route.